

Gọt bỏ rườm rà, giữ lại nét thanh: bàn về việc tận dụng thông tin

Zhang Yifei

Nguồn gốc: Eliminate redundancy and keep the lean form: on making full use of information

IOI China candidate team papers / 2000 / Zhang Yifei.pdf

Tóm tắt

Trong thiết kế thuật toán, người ta thường vô thức thực hiện một lượng lớn tính toán thừa. Những phần rườm rà đó làm giảm mạnh hiệu quả của thuật toán. Tác giả cho rằng tận dụng đầy đủ thông tin đã biết là một cách hữu hiệu để giải quyết vấn đề này.

Tận dụng đầy đủ thông tin nghĩa là trong thiết kế thuật toán, cố gắng dùng triệt để mọi thông tin đã biết, nhờ đó tránh tính toán dư thừa, giảm độ phức tạp thời gian và không gian, và nâng cao hiệu quả thuật toán. Bài viết này thảo luận bước đầu việc tận dụng thông tin trong tối ưu hóa quay lui, quy hoạch động và tính toán số.

Từ khóa. Thông tin; tối ưu hóa thuật toán.

“Gọt bỏ rườm rà, giữ lại nét thanh” [1] vốn nói về vẽ trúc, nhưng chứa một triết lý sâu sắc. Thiết kế thuật toán cũng giống như vẽ trúc: cần gọt bỏ phần rườm rà. Trong thực hành giải bài, ta thường vô thức làm một số tính toán thừa, đồng thời xem nhẹ việc tận dụng đầy đủ thông tin đã biết.

Tận dụng đầy đủ thông tin nghĩa là trong thiết kế thuật toán, cố gắng dùng triệt để mọi thông tin đã biết, để tránh tính toán dư thừa, giảm độ phức tạp thời gian và không gian, từ đó nâng cao hiệu quả thuật toán. Do khuôn khổ có hạn, bài viết chỉ bàn về cách dùng phương pháp này để nâng cao hiệu quả của quay lui, quy hoạch động và tính toán số.

1 Nâng cao hiệu quả của quay lui

Ta biết rằng bản chất của quay lui là quá trình xuất phát từ gốc và duyệt một cây lời giải. Nếu trong cây lời giải có một số cây con có cùng tính chất, thì chỉ cần biết tính chất của một cây con trong số đó, ta có thể suy ra tính chất của các cây con khác. Đây là tư tưởng cơ bản của tìm kiếm có nhớ từ trên xuống [2].

Tìm kiếm có nhớ tránh được một số tính toán thừa, nên hiệu quả hơn tìm kiếm không có nhớ. Nhưng trong một vài tìm kiếm có nhớ, việc sử dụng thông tin vẫn chưa đủ đầy đủ và còn chỗ để tối ưu tiếp.

Ta xét ví dụ sau.

Bài toán đếm quan hệ thứ tự

Dùng hai quan hệ “<” và “=” để sắp xếp tuần tự ba số A, B, C có 13 quan hệ khác nhau:

$$\begin{aligned} A < B < C, \quad A < B = C, \quad A < C < B, \quad A = B < C, \quad A = B = C, \quad A = C < B, \\ B < A < C, \quad B < A = C, \quad B < C < A, \quad B = C < A, \\ C < A < B, \quad C < A = B, \quad C < B < A. \end{aligned}$$

Hãy lập trình tính số quan hệ có thể có khi sắp xếp tuần tự N số.

1.1 Liệt kê mọi biểu thức quan hệ thứ tự

Ta có thể dùng quay lui để liệt kê mọi biểu thức quan hệ thứ tự. Biểu thức quan hệ thứ tự của N số được tạo bởi N chữ cái in hoa và $N - 1$ ký hiệu quan hệ nối các chữ cái. Ta lần lượt liệt kê chữ cái in hoa và ký hiệu quan hệ ở từng vị trí cho tới khi xác định được một biểu thức quan hệ thứ tự.

Vì các biểu thức như $A = B$ và $B = A$ là tương đương, ta quy định chữ cái in hoa đứng trước dấu bằng phải có thứ tự ASCII nhỏ hơn chữ cái đứng sau dấu bằng. Dựa trên ý tưởng này, ta dễ dàng viết được thuật toán quay lui cho bài toán.

Thuật toán 1-1. Tính số quan hệ thứ tự của N số.

```
procedure Count(Step, First, Can);
{ Step: chu cai in hoa thu Step dang duoc xac dinh;
  First: gia tri nho nhat ma chu cai hien tai co the nhan;
  Can: tap cac chu cai in hoa con co the su dung. }
begin
  if Step = N then begin          { xac dinh chu cai cuoi cung }
    for i := First to N do
      if i in Can then Inc(Total); { Total la ket qua dem }
    Exit
  end;
  for i := First to N do          { liet ke chu cai in hoa hien tai }
    if i in Can then begin        { i co the dung }
      Count(Step+1, i+1, Can-[i]); { them dau bang }
      Count(Step+1, 1, Can-[i])    { them dau nho hon }
    end
  end;
end;
```

Sau khi gọi $\text{Count}(1, 1, [1..N])$, giá trị của Total là đáp án. Độ phức tạp thời gian của thuật toán này là $O(N!)$.

1.2 Tận dụng thông tin ở mức thô để tối ưu thuật toán 1-1

Trong thuật toán 1-1 có rất nhiều tính toán dư thừa. Ở cây lời giải khi $N = 3$, ba cây con tương ứng trong hình 1 có hình dạng hoàn toàn như nhau. Một khi đã biết số quan hệ thứ tự sinh ra trong một cây con, ta có thể dùng thông tin đó để trực tiếp biết số quan hệ sẽ sinh ra trong hai cây con còn lại.

Rõ ràng, trong quá trình liệt kê, nếu đã xác định k số đầu tiên và ký hiệu quan hệ tiếp theo là dấu nhỏ hơn, thì số quan hệ thứ tự có thể sinh ra chính là số quan hệ thứ tự có thể sinh ra bởi $N - k$ số còn lại.

Giả sử i số có $F[i]$ quan hệ thứ tự khác nhau. Từ thảo luận trên, trong thuật toán 1-1, sau một lần gọi $\text{Count}(\text{Step}+1, 1, \text{Can}-[i])$, lượng tăng của Total phải là $F[N - \text{Step}]$. Giá trị này có thể được tính trong lần đầu gọi $\text{Count}(\text{Step}+1, 1, \text{Can}-[i])$. Một khi đã biết $F[N - \text{Step}]$, ta có thể dùng $\text{Total} := \text{Total} + F[N - \text{Step}]$ thay cho lời gọi $\text{Count}(\text{Step}+1, 1, \text{Can}-[i])$. Như vậy thu được thuật toán cải tiến 1-2.

Thuật toán 1-2. Tính số quan hệ thứ tự của N số.

```

procedure Count(Step, First, Can);
{ Step, First, Can có ý nghĩa như trong thuật toán 1-1. }
begin
  if Step = N then begin          { xác định chu cái cuối cùng }
    for i := First to N do
      if i in Can then Inc(Total); { Total là kết quả đếm }
    Exit
  end;
  for i := First to N do          { liệt kê chu cái in hoa hiện tại }
    if i in Can then begin        { i có thể dùng }
      Count(Step+1, i+1, Can-[i]); { thêm dấu bang }
      if F[N-Step] = -1 then begin { lan gọi dấu tiên }
        F[N-Step] := Total;
        Count(Step+1, 1, Can-[i]); { thêm dấu nhỏ hơn }
        F[N-Step] := Total - F[N-Step]
      end else
        Total := Total + F[N-Step]
    end
  end;
end;

```

Ban đầu, khởi tạo $F[0], F[1], \dots, F[N-1]$ bằng -1 . Sau khi gọi $\text{Count}(1, 1, [1..N])$, giá trị của Total là đáp án. Thuật toán 1-2 chỉ khác thuật toán 1-1 ở đoạn chương trình dùng mảng F .

Thuật toán 1-2 sử dụng các giá trị $F[0], F[1], \dots, F[N-1]$, nhờ đó sau khi xác định thêm dấu nhỏ hơn, nó tránh được tìm kiếm thừa và nhanh chóng tìm ra số phương án cần thiết. Về bản chất đây là tìm kiếm theo kiểu có nhớ từ trên xuống; độ phức tạp thời gian là $O(2^N)$ [3]. So với thuật toán 1-1, hiệu quả đã được nâng lên, nhưng vẫn chưa thật lý tưởng.

1.3 Tận dụng đầy đủ thông tin để tối ưu tiếp thuật toán 1-2

Trong quá trình tìm kiếm, nếu xác định rằng sau chữ cái in hoa thứ k sẽ thêm dấu nhỏ hơn đầu tiên, ta có được hai thông tin:

1. k chữ cái in hoa phía trước đều được nối bằng dấu bằng.
2. Trên cơ sở đó tiếp tục tìm kiếm sẽ sinh ra $F[N - k]$ biểu thức quan hệ thứ tự.

Như hình 2 trong bản gốc, dấu nhỏ hơn đầu tiên chia biểu thức quan hệ thứ tự thành hai phần. Theo quy tắc nhân, tổng số biểu thức quan hệ thứ tự có dạng này bằng số quan hệ thứ tự mà phần trước có thể sinh ra nhân với số quan hệ thứ tự mà phần sau có thể sinh ra. Thuật toán 1-2 thực chất đã dùng thông tin thứ hai: trực tiếp biết phần sau sinh ra $F[n - k]$ quan hệ thứ tự, còn phần trước vẫn được tính bằng tìm

kiểm. Nhưng nếu dùng thêm thông tin thứ nhất, ta có thể suy ra số quan hệ thứ tự mà phần trước sinh ra chính là số cách chọn k vật trong n vật, tức $\binom{n}{k}$. Do đó ta nhận được công thức truy hồi:

$$F[n] = \sum_{k=1}^n \binom{n}{k} F[n-k], \quad F[0] = 1.$$

Thuật toán dùng công thức này để tính $F[n]$ được gọi là thuật toán 1-3 [4], có độ phức tạp thời gian $O(N^2)$.

1.4 Tổng kết

Bảng sau phân tích hiệu năng của ba thuật toán [5]:

	Mục phân tích	Thuật toán 1-1	Thuật toán 1-2	Thuật toán 1-3
Lý thuyết	Độ phức tạp thời gian	$O(N!)$	$O(2^N)$	$O(N^2)$
	Độ phức tạp không gian	$O(1)$	$O(N)$	$O(N)$
Thực chạy	$N = 7$	1s	< 0.05s	< 0.05s
	$N = 8$	10s	< 0.05s	< 0.05s
	$N = 15$	> 10s	0.5s	< 0.05s
	$N = 17$	> 10s	2s	< 0.05s

Trong quá trình tối ưu thuật toán 1-1, ta dùng thông tin $F[0], F[1], \dots, F[N-1]$ để có thuật toán 1-2, làm độ phức tạp thời gian giảm từ $O(N!)$ xuống $O(2^N)$. Trong thuật toán 1-2, tiếp tục loại bỏ tính toán dư thừa thì thu được thuật toán 1-3 với độ phức tạp $O(N^2)$.

Thuật toán 1-3 tính $F[n]$ thể hiện tư tưởng quy hoạch động. Nói cách khác, việc nâng cao hiệu quả của quay lui bằng cách tận dụng đầy đủ thông tin, về bản chất là biến tìm kiếm thành quy hoạch động.

2 Nâng cao hiệu quả của quy hoạch động

Từ phần trước có thể thấy quy hoạch động hiệu quả chính vì nó tận dụng khá đầy đủ thông tin đã biết. Nhưng trong một số quy hoạch động vẫn tồn tại tính toán dư thừa. Phần này tiếp tục thảo luận cách tận dụng đầy đủ thông tin để nâng cao hiệu quả của quy hoạch động.

Ta xét ví dụ sau.

Bài toán thu nhập lý tưởng

Thu nhập lý tưởng là thu nhập cao nhất có thể đạt được trong giao dịch cổ phiếu với vốn ban đầu 1 đồng, trong đó cho phép mua bán số lượng cổ phiếu không nguyên.

Biết giá mỗi cổ phiếu ở ngày thứ i là $V[i]$ đồng, $1 \leq i \leq M$. Hãy tính thu nhập lý tưởng sau M ngày.

2.1 Một lời giải quy hoạch động

Với bài này, rất dễ nghĩ tới quy hoạch động. Đặt $F[i]$ là thu nhập lớn nhất có thể đạt được khi đóng cửa ngày thứ i . Khi đó:

$$F[i] = \max_{0 \leq j \leq k < i} \left\{ \frac{F[j]}{V[k]} \cdot V[i] \right\}, \quad F[0] = 1, \quad V[0] = 1.$$

Công thức này có nghĩa là: thu nhập lớn nhất có thể đạt được lúc đóng cửa ngày thứ i là lấy toàn bộ thu nhập sau khi đóng cửa ngày thứ j để mua cổ phiếu ngày thứ k , rồi bán toàn bộ cổ phiếu nắm giữ ở ngày

thứ i . Dùng công thức này ta được thuật toán 2-1, có độ phức tạp thời gian $O(M^3)$ và độ phức tạp không gian $O(M)$.

Thuật toán 2-1.

```
F[0] := 1; V[0] := 1; F[1..M] := 0;
for i := 1 to M do
  for j := 0 to i-1 do
    for k := j to i-1 do
      F[i] := Max{F[i], F[j]/V[k]*V[i]}
```

2.2 Đối ý nghĩa biểu diễn trạng thái để tối ưu thuật toán 2-1

Thay đổi ý nghĩa biểu diễn trạng thái trong quy hoạch động là một phương pháp thường dùng để tối ưu quy hoạch động. Với bài này, ta có thể dùng hai cách biểu diễn trạng thái khác nhau để tối ưu thuật toán 2-1.

Cách 1. Đặt $P[i]$ là số cổ phiếu nhiều nhất có thể đạt được trong i ngày đầu. Ta có phương trình chuyển trạng thái:

$$P[i] = \max_{0 \leq j < i} \left\{ P[j], P[j] \cdot \frac{V[j]}{V[i]} \right\}.$$

Lý do là số cổ phiếu nhiều nhất có thể có trong i ngày đầu hoặc là số cổ phiếu nhiều nhất có được trong $i - 1$ ngày đầu, hoặc là bán toàn bộ số cổ phiếu nhiều nhất có được trong j ngày đầu ở ngày thứ j , rồi mua cổ phiếu ngày thứ i . Rõ ràng, số cổ phiếu nhiều nhất trong $i - 1$ ngày đầu nhân với giá cổ phiếu ngày thứ i chính là thu nhập lớn nhất có thể đạt ở ngày thứ i .

Cách 2. Đặt $Q[i]$ là thu nhập lớn nhất có thể đạt được trong i ngày đầu. Ta có:

$$Q[i] = \max_{0 \leq j < i} \left\{ Q[j], \frac{Q[j]}{V[j]} \cdot V[i] \right\}.$$

Nghĩa là thu nhập lớn nhất có thể đạt trong i ngày đầu hoặc là thu nhập lớn nhất có thể đạt trong $i - 1$ ngày đầu, hoặc là thu nhập lớn nhất có được khi mua cổ phiếu ở ngày thứ j rồi bán ở ngày thứ i .

Hai cách trên đều có độ phức tạp thời gian $O(M^2)$. Điều này cho thấy việc thay đổi ý nghĩa biểu diễn trạng thái có thể nâng cao hiệu quả thuật toán ở một mức độ nhất định. Nhưng với bài này, chỉ đổi ý nghĩa trạng thái thì khó tối ưu thêm.

2.3 Tận dụng thông tin ở mức thô để tối ưu thuật toán 2-1

Đoạn lặp trong thuật toán 2-1 có nhiệm vụ xác định giá trị lớn nhất mà $F[i]$ có thể đạt. Vì $V[i]$ không đổi, $F[i]$ đạt lớn nhất khi và chỉ khi $F[j]/V[k]$ đạt lớn nhất, với $0 \leq j \leq k < i$.

Trong thuật toán 2-1, ta dùng hai vòng lặp để xác định giá trị lớn nhất của $F[j]/V[k]$. Nhưng khi xác định giá trị lớn nhất của $F[i - 1]$, thực ra ta đã tìm được giá trị lớn nhất của $F[j]/V[k]$ khi $0 \leq j \leq k < i - 1$. Nếu tận dụng thông tin này, ta có thể xác định giá trị lớn nhất của $F[i]$ nhanh hơn. Theo ý tưởng của hình 3 trong bản gốc, để xác định giá trị lớn nhất của miền mới, chỉ cần so sánh giá trị lớn nhất của miền cũ với giá trị lớn nhất của phần mới thêm vào.

Đặt

$$\text{MaxFV}[i] = \max_{0 \leq j \leq k < i} \left\{ \frac{F[j]}{V[k]} \right\}.$$

Khi đó:

$$\begin{aligned}\text{MaxFV}[i] &= \max \left\{ \max_{0 \leq j \leq k < i-1} \left\{ \frac{F[j]}{V[k]} \right\}, \max_{\substack{0 \leq j \leq k \\ k=i-1}} \left\{ \frac{F[j]}{V[k]} \right\} \right\} \\ &= \max \left\{ \text{MaxFV}[i-1], \max_{0 \leq j \leq i-1} \left\{ \frac{F[j]}{V[i-1]} \right\} \right\}.\end{aligned}$$

Từ công thức của $F[i]$:

$$\begin{aligned}F[i] &= \max_{0 \leq j \leq k < i} \left\{ \frac{F[j]}{V[k]} \cdot V[i] \right\} \\ &= \max_{0 \leq j \leq k < i} \left\{ \frac{F[j]}{V[k]} \right\} \cdot V[i] = \text{MaxFV}[i] \cdot V[i].\end{aligned}$$

Do đó thu được truy hồi:

$$\begin{aligned}F[i] &= \text{MaxFV}[i] \cdot V[i], \\ \text{MaxFV}[i] &= \max_{0 \leq j < i} \left\{ \text{MaxFV}[i-1], \frac{F[j]}{V[i-1]} \right\}.\end{aligned}$$

Từ công thức này có thể thấy, khi xác định $\text{MaxFV}[i]$, ta đã tận dụng khá đầy đủ kết quả sinh ra khi xác định $\text{MaxFV}[i-1]$. Dùng công thức trên thu được thuật toán 2-2, với độ phức tạp thời gian $O(M^2)$ và độ phức tạp không gian $O(M)$.

Thuật toán 2-2.

```
F[0] := 1; MaxFV[0] := 0; V[0] := 1;
for i := 1 to M do begin
  MaxFV[i] := MaxFV[i-1];
  for j := 0 to i-1 do
    MaxFV[i] := Max{MaxFV[i], F[j]/V[i-1]};
  F[i] := MaxFV[i]*V[i]
end;
```

Rút gọn công thức trên:

$$\begin{aligned}\text{MaxFV}[i] &= \max_{0 \leq j < i} \left\{ \text{MaxFV}[i-1], \frac{F[j]}{V[i-1]} \right\} \\ &= \max_{0 \leq j < i} \left\{ \text{MaxFV}[i-1], \text{MaxFV}[j] \cdot \frac{V[j]}{V[i-1]} \right\}.\end{aligned}$$

Trong công thức này, $\text{MaxFV}[i]$ có thể được xem là số cổ phiếu nhiều nhất có thể có trong $i-1$ ngày đầu. Cách biểu diễn trạng thái này về bản chất giống ý nghĩa của $P[i]$ trong công thức cách 1. Như vậy, bằng cách sử dụng thông tin đã biết, ta đạt được hiệu quả tương đương việc thay đổi ý nghĩa biểu diễn trạng thái trong quy hoạch động.

2.4 Tận dụng đầy đủ thông tin để tối ưu tiếp thuật toán 2-2

Trong thuật toán 2-2, nếu tiếp tục sử dụng thông tin, rất dễ nhận được thuật toán $O(M)$.

Đoạn lặp trong thuật toán 2-2 có nhiệm vụ xác định giá trị lớn nhất mà $\text{MaxFV}[i]$ có thể đạt. Vì $V[i-1]$ không đổi, $F[j]/V[i-1]$ đạt lớn nhất khi và chỉ khi $F[j]$ đạt lớn nhất, với $0 \leq j < i$.

Vòng lặp trong của thuật toán 2-2 thực chất là tìm giá trị lớn nhất trong $F[0], F[1], \dots, F[i-1]$ khi xác định $\text{MaxFV}[i]$. Còn khi xác định $\text{MaxFV}[i-1]$, ta đã tìm được giá trị lớn nhất trong $F[0], F[1], \dots, F[i-2]$. Nếu dùng thông tin này, ta có thể xác định giá trị lớn nhất trong $F[0], F[1], \dots, F[i-1]$ nhanh hơn.

Đặt

$$\text{MaxF}[i] = \max_{0 \leq j < i} \{F[j]\}.$$

Khi đó:

$$\begin{aligned} \text{MaxF}[i] &= \max_{0 \leq j < i} \{F[j]\} \\ &= \max \left\{ \max_{0 \leq j < i-1} \{F[j]\}, F[i-1] \right\} \\ &= \max \{ \text{MaxF}[i-1], F[i-1] \}, \\ \text{MaxFV}[i] &= \max_{0 \leq j < i} \left\{ \text{MaxFV}[i-1], \frac{F[j]}{V[i-1]} \right\} \\ &= \max \left\{ \text{MaxFV}[i-1], \frac{\text{MaxF}[i]}{V[i-1]} \right\}. \end{aligned}$$

Do đó:

$$\begin{aligned} F[i] &= \text{MaxFV}[i] \cdot V[i], \\ \text{MaxFV}[i] &= \max \left\{ \text{MaxFV}[i-1], \frac{\text{MaxF}[i]}{V[i-1]} \right\}, \\ \text{MaxF}[i] &= \max \{ \text{MaxF}[i-1], F[i-1] \}. \end{aligned}$$

Từ công thức này có thể thấy, khi xác định $\text{MaxF}[i]$, ta đã tận dụng đầy đủ thông tin sinh ra khi xác định $\text{MaxF}[i-1]$. Dùng công thức trên thu được thuật toán 2-3, có độ phức tạp thời gian $O(M)$. Ba truy hồi chỉ phụ thuộc trạng thái ngay trước đó, nên độ phức tạp không gian có thể giảm tiếp xuống $O(1)$ [6].

Thuật toán 2-3.

```
F := 1; MaxF := 0; MaxFV := 0; V[0] := 1;
for i := 1 to M do begin
  if F > MaxF then MaxF := F;
  if MaxF/V[i-1] > MaxFV then MaxFV := MaxF/V[i-1];
  F := MaxFV*V[i]
end;
```

Trong công thức trên, $\text{MaxF}[i]$ có thể được xem là thu nhập lớn nhất có thể đạt trong $i-1$ ngày đầu [7]. Tuy cách biểu diễn trạng thái này tương tự $Q[i]$, độ phức tạp thời gian lại giảm từ $O(M^2)$ xuống $O(M)$, và độ phức tạp không gian giảm từ $O(M)$ xuống $O(1)$. Như vậy, bằng cách tận dụng đầy đủ thông tin đã biết, ta đạt được hiệu quả tối ưu mà việc chỉ thay đổi biểu diễn trạng thái khó đạt tới.

2.5 Tổng kết

Bảng sau phân tích hiệu năng của ba thuật toán:

	Mục phân tích	Thuật toán 2-1	Thuật toán 2-2	Thuật toán 2-3
Lý thuyết	Độ phức tạp thời gian	$O(M^3)$	$O(M^2)$	$O(M)$
	Độ phức tạp không gian	$O(M)$	$O(M)$	$O(1)$
Thực chạy	$M = 200$ dữ liệu ngẫu nhiên	4s	0.05s	< 0.05s
	$M = 1000$ dữ liệu ngẫu nhiên	> 60s	1s	< 0.05s
	$M = 2000$ dữ liệu ngẫu nhiên	> 60s	5s	< 0.05s

Trong bài toán này, bằng cách tránh tính toán dư thừa, ta đưa độ phức tạp thời gian từ $O(M^3)$ xuống $O(M^2)$, rồi tiếp tục xuống $O(M)$. Độ phức tạp không gian cũng giảm từ $O(M)$ xuống $O(1)$.

Từ đó có thể thấy việc tận dụng đầy đủ thông tin để nâng cao hiệu quả quy hoạch động là rất hữu hiệu. Hơn nữa, tận dụng đầy đủ thông tin không nhất thiết phải đánh đổi bằng không gian; nó cũng có thể tối ưu độ phức tạp không gian.

3 Nâng cao hiệu quả của tính toán số

Qua các phân tích trước, ta cảm nhận sâu sắc tư tưởng cốt lõi của quy hoạch động: tận dụng đầy đủ thông tin đã biết. Nhưng tư tưởng này không chỉ giới hạn trong quy hoạch động. Dưới đây ta sẽ thấy rằng tận dụng đầy đủ thông tin cũng rất hiệu quả trong việc nâng cao hiệu quả tính toán số.

Xét bài toán tính toán số sau.

Bài toán tính toán độ chính xác cao

Bài thi cấp tỉnh Hồ Nam năm 1998: nhập n, m, k , tính k chữ số cuối của

$$S_m(n) = 1^m + 2^m + \dots + n^m,$$

trong đó $1 \leq k \leq 99$ và $1 \leq n, m \leq 9999$.

3.1 Phân tích

Vì k có thể tới 99, cần dùng tính toán độ chính xác cao. Bài toán chỉ yêu cầu k chữ số cuối của $S_m(n)$, nên mỗi lần tính chỉ cần giữ k chữ số cuối của kết quả. Rõ ràng, phần lớn thời gian tính $S_m(n)$ sẽ tiêu tốn vào phép lũy thừa. Sau đây ta phân tích các thuật toán lũy thừa.

3.2 Thuật toán lũy thừa thô sơ

Theo định nghĩa lũy thừa, ta nhân i với chính nó m lần.

Thuật toán 3-1. Tính i^m .

```
Func Power(i, m);
begin
  SetValue(x, 1);      { x là số do chính xác cao }
  for k := 1 to m do
    x := Mul(x, i);     { Mul là phép nhân do chính xác cao,
                        tra về giá trị x*y }
  Power := x
end;
```

Dùng thuật toán 3-1 để tính i^m cần thực hiện m phép nhân độ chính xác cao.

3.3 Tận dụng thông tin ở mức thô để tối ưu thuật toán 3-1

Thuật toán 3-1 sử dụng thông tin rất chưa đầy đủ. Chẳng hạn cần tính i^5 , và hiện đã tính được i^3 . Thuật toán 3-1 sẽ dùng i^3 và i để tính i^4 , rồi tiếp tục tính i^5 . Nhưng thực ra trước khi tính i^3 , ta đã tính được i^2 ; nhân i^3 với i^2 là có i^5 .

Ta dùng mảng $p[k]$ lưu kết quả i^k đã tính, và trong quá trình tính cố gắng sử dụng thông tin đã có. Ví dụ, có thể tính i^{15} như sau:


```

p[1] = i
p[2] = p[1]*p[1]
p[3] = p[2]*p[1]
p[6] = p[3]*p[3]
p[7] = p[6]*p[1]
p[14] = p[7]*p[7]
p[15] = p[14]*p[1]

```

Như vậy chỉ dùng 6 phép nhân độ chính xác cao đã tính được i^{15} .

Từ ví dụ trên có thể thấy, bình phương một kết quả đã biết cho phép dùng một phép nhân để tính được một lũy thừa có số mũ khá lớn. Theo ý tưởng đó ta có thuật toán 3-2:

$$i^m = \begin{cases} (i^{m/2})^2, & m \text{ chẵn}, \\ i(i^{(m-1)/2})^2, & m \text{ lẻ}. \end{cases}$$

Thuật toán 3-2. Tính i^m .

```

Func Power(i, m);
begin
  if m = 1 then Power := i
  else begin
    Temp := Power(i, m div 2);
    Temp := Mul(Temp, Temp);
    if Odd(m) then Power := Mul(Temp, i)
    else Power := Temp
  end
end;

```

Thuật toán 3-2 cần $O(\log_2 m)$ phép nhân độ chính xác cao để tính i^m . Như vậy, thông qua việc tận dụng thông tin ở mức thô [8], độ phức tạp thời gian giảm từ $O(m)$ xuống $O(\log_2 m)$.

Thuật toán 3-2 thực chất là phương pháp chia đôi. Dùng chia đôi để tính lũy thừa hiệu quả hơn nhân lặp chính vì nó tận dụng thông tin đã biết đầy đủ hơn.

3.4 Tận dụng đầy đủ thông tin để tối ưu tiếp thuật toán 3-2

Chú ý rằng khi tính i^m , ta đã biết các giá trị $1^m, 2^m, \dots, (i-1)^m$, nhưng những giá trị này không hề được thuật toán 3-2 dùng đến khi tính i^m . Nếu có thể thiết lập quan hệ giữa i^m với $1^m, 2^m, \dots, (i-1)^m$, ta có thể tính i^m nhanh hơn.

Ví dụ, cần tính 120^m . Lúc này ta đã tính được 5^m và 24^m . Rõ ràng, nhân 5^m với 24^m sẽ thu được 120^m .

Khi i là hợp số, giả sử $i = pq$, trong đó $1 < p, q < i$. Khi đó $i^m = (pq)^m = p^m q^m$. Vì $p, q < i$, khi tính i^m , hai giá trị p^m và q^m đều là thông tin có thể dùng trực tiếp. Vì vậy với hợp số i , ta chỉ cần một phép nhân độ chính xác cao để tính được i^m .

Thuật toán thu được theo cách này gọi là thuật toán 3-3. Giả sử trong $1..n$ có x hợp số, thuật toán 3-3 chỉ thực hiện $(n-x) \log_2 m + x$ phép nhân độ chính xác cao. Số nguyên tố phân bố khá thưa trong các số tự nhiên; thực nghiệm cho thấy khi $n = m = 9999, k = 99$, hiệu quả của thuật toán 3-3 gấp 5 lần thuật toán 3-2.

3.5 Tổng kết

Bảng sau phân tích hiệu năng của ba thuật toán:

	Mục phân tích	Thuật toán 3-1	Thuật toán 3-2	Thuật toán 3-3
Lý thuyết	Độ phức tạp thời gian	$O(nm)$	$O(n \log_2 m)$	$O(n \log_2 m)$
	Độ phức tạp không gian	$O(k)$	$O(k)$	$O(nk)$
Thực chạy	$N = M = 100, K = 99$	0.1s	< 0.05s	< 0.05s
	$N = 100, M = 9999, K = 99$	20s	0.5s	0.1s
	$N = M = 1000, K = 99$	15s	3s	1s
	$N = M = 9999, K = 99$	> 60s	55s	10s

Trong ví dụ này, trước hết ta tận dụng thông tin ở mức thô để tối ưu thuật toán lũy thừa thô sơ 3-1, thu được thuật toán 3-2 dùng phương pháp chia đôi. Phân tích tiếp cho thấy khi tính i^m , phương pháp chia đôi không dùng các thông tin đã tính như $1^m, 2^m, \dots, (i-1)^m$. Tận dụng đầy đủ các thông tin này cho ta thuật toán nhanh hơn 3-3. Vì thuật toán 3-3 cần lưu một số giá trị đã tính [9], về bản chất nó đánh đổi không gian lấy thời gian.

4 Kết luận

Thêm thông tin ghi nhớ vào tìm kiếm để nâng cao hiệu quả quay lui, thay đổi ý nghĩa biểu diễn trạng thái trong quy hoạch động để nâng cao hiệu quả quy hoạch động, và dùng phương pháp chia đôi để nâng cao hiệu quả tính toán số, tất cả đều thể hiện ở những mức độ khác nhau việc tận dụng đầy đủ thông tin đã biết. Nhưng nếu chỉ dùng các kỹ thuật thường gặp này để tối ưu thuật toán, vẫn có thể còn tính toán thừa. Nếu có thể tận dụng thông tin đã biết đầy đủ hơn nữa, ta sẽ thu được hiệu quả tốt hơn.

Muốn gọt bỏ phần rườm rà trong thuật toán, điểm then chốt là tìm ra các thông tin đã biết có thể sử dụng. Một khi tận dụng đầy đủ chúng, hiệu quả thuật toán có thể được nâng lên rất nhiều.

Tài liệu tham khảo

1. *Olympic Tin học*, 1999(3).
2. Wu Wenhui, Wang Jiande. *Thuật toán thực dụng và thiết kế chương trình*. Nhà xuất bản Công nghiệp Điện tử, 1998.01.
3. Liu Fusheng, Wang Jiande. *Hướng dẫn thi Olympic Tin học (máy tính) quốc tế cho thanh thiếu niên: tìm kiếm trí tuệ nhân tạo và thiết kế chương trình*. Nhà xuất bản Công nghiệp Điện tử, 1993.04.
4. *Tập Zheng Banqiao*.

Phụ lục

[1]. Trích từ trang 206 của *Tập Zheng Banqiao*. Toàn bài thơ như sau:

Đề tranh trúc
Bốn mươi năm vẽ hành trúc
Ngày vung bút viết, đêm ngẫm suy
Gọt bỏ rườm rà, giữ lại nét thanh
Vẽ đến khi tường lạ mới là lúc đã quen.

[2]. Xem tài liệu tham khảo [2].

[3]. Trong phân tích phía sau có thể thấy tổng số lần gọi đoạn chương trình được tối ưu là

$$\sum_{i=0}^n \sum_{j=0}^i \binom{i}{j} = \sum_{i=0}^n 2^i = 2^{n+1} - 1.$$

Vì vậy độ phức tạp thời gian của thuật toán là $O(2^{n+1}) = O(2^n)$.

[4]. Cũng có thể trực tiếp suy ra công thức 1 bằng phương pháp toán học. Nhưng tôi cho rằng cách đó khó nắm hơn phương pháp dùng trong bài này. Trong tài liệu tham khảo [1] có cho một công thức có hình thức đơn giản hơn, nhưng việc thiết kế công thức đó cũng khá khó. Công thức trong tài liệu tham khảo [1] là:

$$\begin{aligned} F(m, t) &= [F(m-1, t-1) + F(m-1, t)] \cdot t, \\ F(1, 1) &= 1, \quad F(1, t) = 0 \quad (t > 0), \\ \text{số quan hệ thứ tự của } n \text{ số} &= \sum_{i=1}^n F(n, i). \end{aligned}$$

[5]. Môi trường chạy của mọi chương trình trong bài là Pentium 100MHz, biên dịch bằng BP7.0.

[6]. Nếu vừa đọc dữ liệu vừa quy hoạch, ta chỉ cần lưu $V[i-1]$ và $V[i]$, mà không cần định nghĩa mảng V độ dài M . Nhờ vậy độ phức tạp không gian của thuật toán giảm xuống $O(1)$.

[7]. Công thức 6 còn có thể rút gọn tiếp:

$$\begin{aligned} \text{MaxF}[i] &= \max\{\text{MaxF}[i-1], F[i-1]\} \\ &= \max\{\text{MaxF}[i-1], \text{MaxFV}[i-1] \cdot V[i-1]\}, \\ \frac{\text{MaxF}[i]}{V[i-1]} &= \frac{\max\{\text{MaxF}[i-1], \text{MaxFV}[i-1] \cdot V[i-1]\}}{V[i-1]} \\ &= \max\left\{\frac{\text{MaxF}[i-1]}{V[i-1]}, \text{MaxFV}[i-1]\right\} \\ &\geq \text{MaxFV}[i-1], \\ \text{MaxFV}[i] &= \max\left\{\text{MaxFV}[i-1], \frac{\text{MaxF}[i]}{V[i-1]}\right\} = \frac{\text{MaxF}[i]}{V[i-1]}, \\ \text{MaxF}[i] &= \max\left\{\text{MaxF}[i-1], \frac{\text{MaxF}[i-1]}{V[i-2]} \cdot V[i-1]\right\}. \end{aligned}$$

Do đó có thể nhận được truy hồi:

$$\text{MaxF}[i] = \max\left\{\text{MaxF}[i-1], \frac{\text{MaxF}[i-1]}{V[i-2]} \cdot V[i-1]\right\}.$$

Trong chương trình nguồn Pro_2.pas, tác giả đưa ra thuật toán 2-4 cho công thức này. Độ phức tạp thời gian và không gian của thuật toán 2-4 giống thuật toán 2-3, chỉ là hình thức đơn giản hơn.

[8]. Cần nói thêm rằng dùng phương pháp chia đôi để tính lũy thừa không phải là cách tận dụng thông tin đầy đủ nhất. Ví dụ, dùng chia đôi để tính i^{15} cần 6 phép nhân, trong khi thực ra chỉ cần 5 phép nhân:

```
p[1] = i;  
p[2] = p[1]*p[1]  
p[3] = p[2]*p[1]  
p[6] = p[3]*p[3]  
p[9] = p[6]*p[3]  
p[15] = p[9]*p[6]
```

Muốn có phương án tối ưu cho lũy thừa cần tốn rất nhiều thời gian, xem tài liệu tham khảo [3]. Vì số lần tính của phương pháp chia đôi rất gần số lần tối ưu, bài viết chọn phương pháp chia đôi.

[9]. Ta chỉ cần lưu các giá trị đã tính $1^m, 2^m, \dots, 4999^m$. Tất nhiên, còn có thể tối ưu thêm độ phức tạp không gian; xem chương trình Pro_3.pas để biết chi tiết.